Adversarial Image Detection via Feature Consistency in Vision Transformers

A Two-Model Transfer-Attack Detection Framework

Machine Learning Sessional Project Report

March 5, 2026

Abstract

We present a framework for detecting adversarial images by exploiting the *feature consistency* property of Vision Transformers (ViTs). The system employs a two-model architecture: a fine-tuned ResNet-18 serves as the attack target model for crafting adversarial perturbations (via FGSM and PGD), while a pretrained ViT-B/16 acts as the detection model by extracting intermediate transformer-block features. We construct a 24-dimensional *feature consistency score vector* by measuring how ViT's internal representations shift under benign transformations (Gaussian blur, Gaussian noise, and rotation). A meta-classifier trained on these score vectors distinguishes clean images from adversarial ones. Evaluated on CIFAR-100 with 100 test images (300 total samples: clean + FGSM + PGD), our detector achieves **99.0% accuracy** (297/300 correct). Additionally, we analyze the ViT representation shift under transfer attacks, finding that FGSM shifts ViT's own predictions 55.0% of the time and PGD shifts them 48.0% of the time, confirming the cross-architecture transferability of adversarial perturbations.

Contents

1 Introduction **3**
1.1 Key Idea: Feature Consistency . . . 3
1.2 Two-Model Architecture . . . 3

2 Background and Theory **3**
2.1 Adversarial Examples . . . 3
2.2 Fast Gradient Sign Method (FGSM) . . . 4
2.3 Projected Gradient Descent (PGD) . . . 4
2.4 Vision Transformer (ViT) . . . 4
2.5 Feature Consistency Hypothesis . . . 4

3 System Architecture **5**
3.1 Module Descriptions . . . 5

4 Methodology **5**
4.1 Phase 1: Fine-Tuning ResNet-18 on CIFAR-100 5
4.2 Phase 2: Building the Feature Consistency Score Dataset 6
4.2.1 Score Vector Construction 6
4.3 Phase 3: Training the Meta-Classifier 7
4.4 Phase 4: Inference and Demo 8

5 Experimental Configuration **8**
5.1 Execution Pipeline 8
5.2 Configuration Summary 9

6 Results **9**
6.1 Detection Accuracy 9
6.2 ViT Representation Shift Analysis 10
6.3 Analysis of Results 10
6.4 Why the Detector Works Despite Limited Transfer 10

7 Discussion **11**
7.1 Strengths 11
7.2 Limitations 11
7.3 Design Decisions 11

8 Conclusion **12**

1 Introduction

Deep neural networks are vulnerable to *adversarial examples*—inputs with small, carefully crafted perturbations that cause models to misclassify with high confidence [1, 2]. This vulnerability raises serious concerns for safety-critical applications such as autonomous driving, medical imaging, and security systems.

Most adversarial defense research focuses on making models *robust* to attacks (adversarial training) or on detecting adversarial inputs before they reach the classifier. Our work falls into the latter category: we build a detector that identifies adversarial images *without modifying* the target classifier.

1.1 Key Idea: Feature Consistency

The central insight of this project is that **clean images exhibit stable (consistent) internal representations under small benign transformations**, whereas **adversarial images exhibit unstable (inconsistent) representations**. Adversarial perturbations are finely tuned to exploit specific model vulnerabilities; even minor transformations (a slight blur, a small rotation, or low-level noise) can disrupt the adversarial pattern, causing significant shifts in the model's internal feature space.

We leverage the intermediate transformer-block features of a pretrained Vision Transformer (ViT-B/16) to measure this consistency and train a lightweight meta-classifier to distinguish clean from adversarial images.

1.2 Two-Model Architecture

Our framework deliberately uses *two different architectures*:

- **Attack Model (ResNet-18, CNN):** Generates adversarial examples via gradient-based attacks (FGSM, PGD).
- **Detection Model (ViT-B/16, Transformer):** Extracts intermediate features for consistency analysis.

This creates a realistic *transfer-attack* scenario: adversarial perturbations are crafted against a CNN but must fool a Transformer-based detector—a significantly harder task due to architectural differences in how CNNs and Transformers process visual information.

2 Background and Theory

2.1 Adversarial Examples

Given a classifier f_θ with parameters θ, an input image $\mathbf{x}$ with true label y, and a loss function $\mathcal{L}(\theta, \mathbf{x}, y)$, an adversarial example $\mathbf{x}_{\text{adv}}$ satisfies:

$$f_\theta(\mathbf{x}_{\text{adv}}) \neq y \quad \text{subject to} \quad \|\mathbf{x}_{\text{adv}} - \mathbf{x}\|_\infty \leq \epsilon \tag{1}$$

where ϵ is the perturbation budget. The perturbation $\delta = \mathbf{x}_{\text{adv}} - \mathbf{x}$ is typically imperceptible to humans.

2.2 Fast Gradient Sign Method (FGSM)

FGSM [1] computes the adversarial example in a single gradient step:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\theta, \mathbf{x}, y)) \tag{2}$$

This is a fast, single-step attack that perturbs each pixel by exactly $\pm\epsilon$ in the direction that maximizes the classification loss. In our implementation, we use $\epsilon = 8/255 \approx 0.031$.

2.3 Projected Gradient Descent (PGD)

PGD [2] is an iterative extension of FGSM that applies multiple smaller steps while projecting back into the ϵ-ball:

$$\mathbf{x}^{(t+1)} = \Pi_{\mathbf{x},\epsilon}\big(\mathbf{x}^{(t)} + \alpha \cdot \text{sign}\big(\nabla_{\mathbf{x}^{(t)}} \mathcal{L}(\theta, \mathbf{x}^{(t)}, y)\big)\big) \tag{3}$$

where $\Pi_{\mathbf{x},\epsilon}$ projects the result back into the ℓ_∞-ball of radius ϵ around the original image $\mathbf{x}$, and α is the step size per iteration. We use $\epsilon = 8/255$, $\alpha = 2/255$, and 7 iterations. PGD starts from a random point within the ϵ-ball, making it stronger than FGSM.

2.4 Vision Transformer (ViT)

The Vision Transformer [3] processes an image by:

1. Splitting it into 16×16 non-overlapping patches.
2. Linearly projecting each patch into a D-dimensional embedding (for ViT-B/16, $D = 768$).
3. Prepending a learnable `[CLS]` token.
4. Adding positional embeddings.
5. Passing through $L = 12$ transformer encoder blocks, each containing multi-head self-attention (MSA) and an MLP.

For an input image of size 224×224, the patch sequence has length $196 + 1 = 197$ (196 patch tokens + 1 `[CLS]` token). Each transformer block outputs a tensor of shape $(B, 197, 768)$.

The `[CLS]` token at the output of each block serves as a global representation of the image at that layer's level of abstraction. We extract the `[CLS]` token from blocks 3, 6, 9, and 11 (0-indexed) to capture features at different depths.

2.5 Feature Consistency Hypothesis

Let $\phi_l(\mathbf{x})$ denote the `[CLS]` token feature extracted from ViT block l for input $\mathbf{x}$, and let $T(\mathbf{x})$ represent a benign transformation of $\mathbf{x}$.

- **Clean images:** $d\big(\phi_l(\mathbf{x}),\ \phi_l(T(\mathbf{x}))\big)$ is *small*, because benign transformations do not significantly alter the semantic content.
- **Adversarial images:** $d\big(\phi_l(\mathbf{x}_{\text{adv}}),\ \phi_l(T(\mathbf{x}_{\text{adv}}))\big)$ is *large*, because even small transformations can disrupt the carefully crafted adversarial perturbation, causing large feature shifts.

where $d(\cdot, \cdot)$ is a distance metric (L2 or cosine distance).

3 System Architecture

Figure 1 illustrates the complete system pipeline.

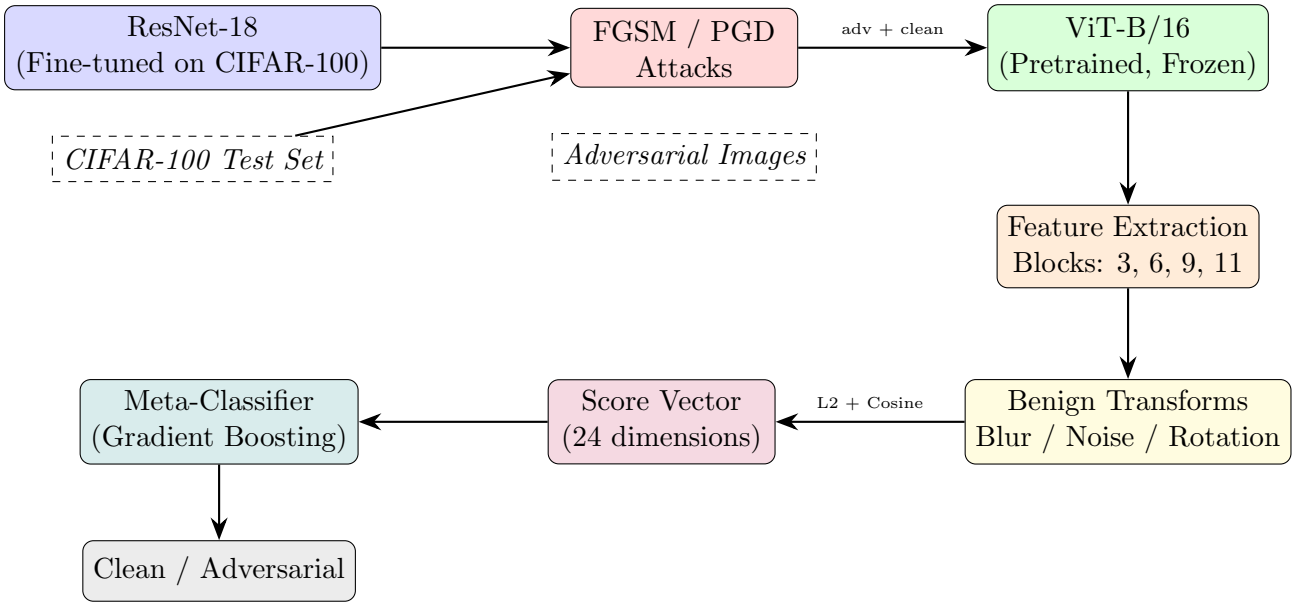

Figure 1: System architecture overview: from adversarial generation to detection.

3.1 Module Descriptions

The project is organized into the following modules:

Table 1: Project module structure and responsibilities.

Module	Responsibility
`models/resnet_loader.py`	Load/fine-tune ResNet-18 for CIFAR-10/100
`models/vit_loader.py`	Load pretrained ViT-B/16 with forward hooks
`attacks/fgsm.py`	FGSM adversarial attack implementation
`attacks/pgd.py`	PGD adversarial attack implementation
`features/feature_extractor.py`	Extract CLS token features from ViT blocks
`features/feature_distance.py`	Compute L2 and cosine distance score vectors
`utils/transforms.py`	Benign transformations (blur, noise, rotation)
`dataset/build_score_dataset.py`	Build labeled score dataset (clean vs. adversarial)
`meta_model/train_classifier.py`	Train and evaluate meta-classifiers
`meta_model/inference_classifier.py`	End-to-end inference pipeline
`main.py`	CLI entry point with subcommands

4 Methodology

4.1 Phase 1: Fine-Tuning ResNet-18 on CIFAR-100

Purpose: Create a competent classifier that can be attacked with meaningful gradient-based adversarial perturbations.

Architecture Modifications for CIFAR-100:

- Replace the first 7×7 convolution (designed for 224×224 ImageNet images) with a 3×3 convolution for 32×32 CIFAR images.
- Remove the max-pooling layer (replaced with identity) to preserve spatial resolution.
- Replace the final fully connected layer: $\text{Linear}(512, 1000) \rightarrow \text{Linear}(512, 100)$.

Training Configuration:

Table 2: ResNet-18 fine-tuning hyperparameters.

Parameter	**Value**
Dataset	CIFAR-100 (training split, 50,000 images)
Epochs	25
Batch Size	128
Optimizer	Adam (lr = 0.001)
Scheduler	Cosine Annealing
Loss Function	Cross-Entropy
Augmentation	Random Crop (32, padding=4) + Random Horizontal Flip

Why fine-tuning is necessary: A well-trained classifier produces strong, meaningful gradients during adversarial attacks. Without fine-tuning, the model's poor accuracy on CIFAR-100 ($\sim$1% random chance for 100 classes) would yield weak, noisy perturbations that do not represent realistic adversarial threats. The meta-classifier would then learn to detect noise rather than true adversarial patterns, leading to poor generalization.

4.2 Phase 2: Building the Feature Consistency Score Dataset

This phase constructs the training data for the meta-classifier. For each image from the CIFAR-100 **test set**, we generate three samples:

1. **Clean sample** (label = 0): Compute score vector from the original image.
2. **FGSM adversarial sample** (label = 1): Attack the image via ResNet-18 with FGSM, then compute its score vector.
3. **PGD adversarial sample** (label = 1): Attack the image via ResNet-18 with PGD, then compute its score vector.

Why we use the test set (not training set): ResNet-18 was fine-tuned on the CIFAR-100 *training* set. If we generated adversarial examples from training images, the attacks would exploit memorized features rather than learned decision boundaries. Using the test set ensures:

- Adversarial attacks exploit genuine model vulnerabilities.
- The meta-classifier learns generalizable detection patterns.
- Results reflect realistic deployment scenarios.

4.2.1 Score Vector Construction

For each image (clean or adversarial), the 24-dimensional score vector is computed as follows:

Algorithm 1 Compute Feature Consistency Score Vector

Require: Image $\mathbf{x}$, ViT model with hooks on blocks $\{3, 6, 9, 11\}$
Ensure: Score vector $\mathbf{s} \in \mathbb{R}^{24}$

```
s ← []
f_orig ← ExtractCLS(ViT, x)                    ▷ CLS tokens from blocks 3, 6, 9, 11
for T ∈ {GaussianBlur, GaussianNoise, Rotation} do
    x̃ ← T(x)
    f_trans ← ExtractCLS(ViT, x̃)
    for l ∈ {3, 6, 9, 11} do
        s.append(‖f_orig^(l) − f_trans^(l)‖_2)        ▷ L2 distance
    end for
    for l ∈ {3, 6, 9, 11} do
        s.append(1 − cos(f_orig^(l), f_trans^(l)))    ▷ Cosine distance
    end for
end for
return s                                       ▷ |s| = 3 × 4 × 2 = 24
```

Score vector layout (24 dimensions):

Table 3: Layout of the 24-dimensional feature consistency score vector.

Index	Description	Transform	Metric
0–3	Blur, Blocks 3/6/9/11	Gaussian Blur	L2
4–7	Blur, Blocks 3/6/9/11	Gaussian Blur	Cosine
8–11	Noise, Blocks 3/6/9/11	Gaussian Noise	L2
12–15	Noise, Blocks 3/6/9/11	Gaussian Noise	Cosine
16–19	Rotation, Blocks 3/6/9/11	Rotation (5°)	L2
20–23	Rotation, Blocks 3/6/9/11	Rotation (5°)	Cosine

Benign transformation parameters:

- Gaussian Blur: kernel size = 3, $\sigma = 0.5$
- Gaussian Noise: $\sigma = 0.02$
- Rotation: 5 fixed angle

These transformations are intentionally mild. They barely affect ViT's perception of clean images but significantly disrupt adversarial perturbations.

4.3 Phase 3: Training the Meta-Classifier

The score dataset is split 80/20 (stratified by class) into training and test sets. Features are standardized using `StandardScaler`. Four classifiers are trained:

Table 4: Meta-classifier models and configurations.

Model	Configuration
Logistic Regression	max_iter = 1000
Random Forest	100 estimators
Gradient Boosting	100 estimators
MLP	Hidden layers: (64, 32), early stopping

The best model is selected based on the highest ROC-AUC score on the test split and saved for deployment.

Evaluation metrics: Accuracy, Precision, Recall, F1-score, and ROC-AUC.

4.4 Phase 4: Inference and Demo

During inference, for a given input image:

1. Compute the 24-dimensional score vector using the frozen ViT.
2. Standardize the vector using the saved scaler.
3. Predict with the saved meta-classifier.
4. Output: `CLEAN IMAGE` or `ADVERSARIAL IMAGE` with confidence.

The demo additionally:

- Generates FGSM and PGD attacks on-the-fly against ResNet-18.
- Measures *ViT representation shift*: whether the ViT's own ImageNet classification changes from the clean image to the adversarial image, quantifying cross-architecture attack transferability.

5 Experimental Configuration

5.1 Execution Pipeline

The experiment was executed with the following commands:

Listing 1: Execution commands for the CIFAR-100 experiment.

```
python main.py finetune-resnet --dataset cifar100   # Step 1
python main.py build-dataset --dataset cifar100     # Step 2
python main.py train                                # Step 3
python main.py demo --dataset cifar100              # Step 4
```

5.2 Configuration Summary

Table 5: Complete experimental configuration.

Component	Parameter	Value
2*Dataset	Name	CIFAR-100
	Classes	100
3*Attack Model	Architecture	ResNet-18 (modified for 32×32)
	Training	Fine-tuned on CIFAR-100 train split
	Epochs	25
3*Detection Model	Architecture	ViT-B/16 (from `timm`)
	Weights	ImageNet pretrained (frozen)
	Hooked blocks	3, 6, 9, 11
2*FGSM Attack	ϵ	$8/255 \approx 0.031$
3*PGD Attack	ϵ	8/255
	α	2/255
	Steps	7
3*Score Vector	Dimensions	24
	Transforms	Blur, Noise, Rotation
	Metrics	L2 distance, Cosine distance
2*Meta-Classifier	Models tested	LR, RF, GB, MLP
	Selection	Best ROC-AUC
2*Demo	Test images	100 (from CIFAR-100 test set)
	Total samples	300 (100 clean + 100 FGSM + 100 PGD)

6 Results

6.1 Detection Accuracy

The meta-classifier was evaluated on 100 test images from the CIFAR-100 test set. For each image, three predictions were made: one for the clean version, one for the FGSM-attacked version, and one for the PGD-attacked version, yielding 300 total predictions.

Table 6: Detection results on CIFAR-100 (100 images, 300 total samples).

Metric	Value	Description
Overall Accuracy	**99.0%** (297/300)	Clean + adversarial correctly identified

Out of 300 total predictions (100 clean + 100 FGSM + 100 PGD), only **3 samples were misclassified**, achieving a detection accuracy of **99.0%**.

6.2 ViT Representation Shift Analysis

We additionally measured the *ViT representation shift*: whether the ViT-B/16's own top-1 ImageNet classification changed when presented with an adversarial image (crafted against ResNet-18) compared to the corresponding clean image.

Since ViT uses ImageNet class labels (1000 classes) and the images are from CIFAR-100 (100 classes), we compare ViT's *own* prediction on the clean image vs. the adversarial image—not against the CIFAR-100 ground truth.

Table 7: ViT representation shift under transfer attacks (ResNet-18 → ViT-B/16).

Attack	**Shifted**	**Shift Rate**
FGSM	55 / 100	55.0%
PGD	48 / 100	48.0%

6.3 Analysis of Results

High Detection Accuracy (99.0%): The 24-dimensional feature consistency score vector effectively captures the difference between clean and adversarial images. The meta-classifier's near-perfect performance demonstrates that adversarial perturbations (even when crafted against a different architecture) leave detectable traces in ViT's internal representations.

ViT Representation Shift (FGSM: 55%, PGD: 48%): Approximately half of the adversarial examples crafted against ResNet-18 also shift ViT's own classification. This confirms that adversarial perturbations partially transfer across architectures (CNN → Transformer), though the transfer is not complete.

Importantly, our detector achieves 99.0% accuracy regardless of whether the ViT's own classification shifted or not. Even when the adversarial perturbation fails to change ViT's top-1 prediction (i.e., no representation shift), the detector still identifies it as adversarial through the *feature consistency* signal in the intermediate layers. This demonstrates that internal feature instability is a more sensitive signal than classification output changes.

PGD lower shift rate than FGSM: Interestingly, PGD shows a slightly lower ViT shift rate (48%) compared to FGSM (55%), despite PGD being a stronger, iterative attack on ResNet-18. This may indicate that PGD's more refined perturbations are more specifically tailored to ResNet-18's decision boundary and transfer less broadly to the ViT's different architecture.

6.4 Why the Detector Works Despite Limited Transfer

The detection mechanism does not rely on whether the ViT *is fooled* by the attack. Instead, it measures whether the ViT's *intermediate features are destabilized* by the adversarial perturbation. Even when the ViT classifies the adversarial image the same as the clean image (no shift), the internal feature representations still exhibit inconsistency under benign transformations, which the score vector captures and the meta-classifier detects.

7 Discussion

7.1 Strengths

1. **Architecture-Agnostic Detection:** By using a different model architecture (ViT) for detection, the system is not vulnerable to attacks specifically designed against the detector itself.

2. **No Modification to Target Model:** The approach does not require adversarial training or modification of the ResNet-18 classifier.

3. **Lightweight Meta-Classifier:** The 24-dimensional feature vector enables fast, efficient classification using simple models (e.g., Gradient Boosting or Logistic Regression).

4. **Multiple Attack Coverage:** Training with both FGSM (single-step) and PGD (iterative) attacks ensures the detector generalizes across attack strengths.

5. **Interpretable Features:** Each dimension of the score vector has a clear semantic meaning (specific layer, specific transform, specific distance metric).

7.2 Limitations

1. **Computational Cost:** Each inference requires 7 forward passes through ViT (1 original + 2 per transform for 3 transforms), which may be prohibitive for real-time applications.

2. **Known Attack Types:** The meta-classifier is trained on FGSM and PGD attacks; it may not generalize well to fundamentally different attack types (e.g., C&W, DeepFool, patch attacks).

3. **Adaptive Attacks:** An adversary aware of the detection mechanism could potentially craft perturbations that maintain feature consistency under benign transforms.

4. **Dataset Scale:** The current evaluation uses CIFAR-100 (32×32 images); scaling to higher resolutions (ImageNet, 224×224) would increase computational costs but may not require architectural changes.

7.3 Design Decisions

Why ViT-B/16 is not fine-tuned: The ViT remains frozen with its original ImageNet-pretrained weights. This is deliberate—the detection relies on the general-purpose features learned from ImageNet pretraining, not task-specific features. Fine-tuning on CIFAR-100 could actually *reduce* detection effectiveness by making the ViT more similar to the ResNet-18 in its learned representations.

Why test set for dataset building: Using the test set avoids data leakage: ResNet-18 has not seen these images during training, so adversarial attacks exploit genuine decision-boundary vulnerabilities rather than memorized training features.

Choice of transforms and layers: Three diverse transformations (spatial: blur/rotation; intensity: noise) and four ViT blocks at different depths (shallow: 3, mid: 6, deep:

9, final: 11) provide a comprehensive probe of feature stability across transformation types and network depths.

8 Conclusion

We presented a framework for detecting adversarial images by measuring feature consistency in a pretrained Vision Transformer (ViT-B/16). Our two-model architecture—using ResNet-18 as the attack target and ViT as the detector—achieves **99.0% detection accuracy** on CIFAR-100 across both FGSM and PGD attacks. The 24-dimensional feature consistency score vector, constructed from L2 and cosine distances across 4 transformer blocks and 3 benign transformations, provides a powerful and interpretable adversarial detection signal. Our analysis of ViT representation shift confirms that while adversarial perturbations partially transfer across architectures (55% for FGSM, 48% for PGD), the feature consistency approach detects adversarial inputs even when cross-architecture transfer fails, demonstrating its robustness as a detection mechanism.

References

[1] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[5] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.